

Inertia 2 Unleashed — Study Notes

A structured deep dive into the five new performance primitives Inertia 2 introduces: polling, prefetching, deferred props, lazy loading, and infinite scrolling.

1. Executive Summary

Inertia 2 is not a rewrite. The mental model — server returns props, client swaps in a Vue component — is unchanged. What Inertia 2 adds is a set of **performance primitives** that let you opt out of the "send everything in one round trip" default whenever it gets in your way.

Five features make up the bulk of the new surface area:

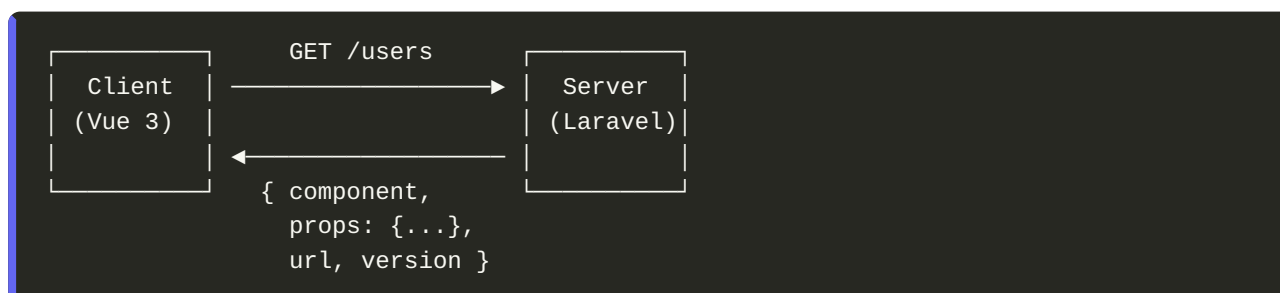
- **Polling** (`usePoll`) — refresh selected props on a timer, ideal for dashboards and live counts.
- **Prefetching** (`prefetch` on `<Link>`) — warm a client-side cache on hover or mount so the next visit feels instant, with a stale-while-revalidate strategy.
- **Deferred props** (`Inertia::defer()` + `<Deferred>`) — return the page immediately, then resolve slow props in a follow-up request, with optional grouping for parallel resolution.
- **Lazy loading** (`Inertia::optional()` + `<WhenVisible>`) — skip a prop entirely until the client explicitly asks, typically when the element scrolls into view.
- **Infinite scrolling** — `<WhenVisible>` plus `Inertia::merge()` plus Laravel's paginator, producing append-on-scroll lists without writing a custom fetch layer.

These features compose. Infinite scroll is literally lazy loading plus prop merging. Prefetching layers on top of partial reloads. The shared backbone is the **partial reload** — Inertia's ability to refetch only specified props (`only: [...]`) and reuse the rest from the current page.

Read this guide once for the mental model, then return to individual episodes when you need the API surface for a specific feature.

2. Mental Model

How a classic Inertia visit works



A single round trip returns **the whole page payload** — every prop the component declares, even if 80% of it doesn't change between visits. That's fine for most pages. It hurts when:

- One prop is slow (a heavy aggregation, a third-party API call).
- One prop is huge (paginated lists, large tables).
- You want the page to feel "alive" without a manual refresh.

What Inertia 2 changes

Inertia 2 keeps the single-round-trip default but introduces escape hatches along two axes:

Axis	Eager (default)	Lazy
Replace	Full page reload	<code>usePoll</code> / partial reload
Merge	n/a	<code>Inertia::merge()</code> (append)
Server-led	Full payload	<code>Inertia::defer()</code> (auto follow-up)
Client-led	Full payload	<code>Inertia::optional()</code> + <code><WhenVisible></code>

Two intuitions to internalize:

1. **Partial reloads are the backbone.** Almost every new feature is a fancy wrapper around "do an Inertia visit but only request these specific props." Polling, prefetch, defer, and `WhenVisible` all reduce to partial reloads under the hood.
2. **The client now drives some of the loading.** In Inertia 1, the server decided what to send. In Inertia 2, the client can say "ask for this prop when the element hits the viewport," or "warm the cache before I click that link." This is the cultural shift.

A small ASCII diagram of where each feature sits:

	replace data	merge data
eager	[full visit]	—
lazy/srv	<code>defer()</code>	<code>merge()</code> (rare alone)
lazy/clt	<code>usePoll</code> , <code>optional+</code> <code>WhenVisible</code>	<code>merge()</code> + <code>WhenVisible</code> (= infinite scroll)

3. Per-Episode Deep Dive

Episode 1 — Polling with `usePoll`

What problem it solves

You have a dashboard showing "users online" or "open tickets." The number changes constantly. You want the UI to reflect reality without making the user refresh. In Inertia 1 you'd reach for `setInterval` and `router.reload({ only: [...] })`. Inertia 2 wraps that pattern in a hook.

Core API surface

```
import { usePoll } from '@inertiajs/vue3'

const { start, stop } = usePoll(
  interval,           // milliseconds between polls (e.g. 2000)
  reloadOptions,      // standard router.reload options (only, onSuccess, ...)
  pollOptions         // { autoStart: boolean, keepAlive: boolean }
)
```

- `interval` — the polling frequency in ms.
- `reloadOptions` — the same shape passed to `router.reload`: `only`, `except`, `onStart`, `onSuccess`, `onError`, `onFinish`, `preserveScroll`, `preserveState`. The `only` array is essentially mandatory; without it you re-fetch the whole page every tick.
- `pollOptions.autoStart` — defaults to `true`. Set `false` if you want to wire a button to start polling manually via the returned `start()` handle.
- `pollOptions.keepAlive` — by default polling stops when the tab is hidden. Set `true` to keep firing in background tabs (use sparingly).

Returns `{ start, stop }` so you can imperatively control the cycle.

Annotated code walkthrough

Server side — a dashboard with a live "users online" count:

```
// routes/web.php
Route::get('/dashboard', function () {
    return Inertia::render('Dashboard', [
        // Wrap the live value in a closure. This is the key detail.
        'usersOnline' => fn () => User::whereNotNull('last_seen_at')
            ->where('last_seen_at', '>', now()->subMinutes(5))
            ->count(),

        // Static-ish data: also closure-wrapped so it doesn't re-run on every poll.
        'totalRevenue' => fn () => Order::sum('total'),
    ]);
});
```

Client side:

```
<script setup>
import { usePoll } from '@inertiajs/vue3'

defineProps({
  usersOnline: Number,
  totalRevenue: Number,
})

// Poll every 2 seconds, but only request the `usersOnline` prop.
// Everything else stays cached from the initial visit.
usePoll(2000, {
  only: ['usersOnline'],
})
</script>

<template>
  <div>
    <h1>Dashboard</h1>
    <p>Users online: {{ usersOnline }}</p>
    <p>Total revenue: ${{ totalRevenue }}</p>
  </div>
</template>
```

Conceptual model

The single most important thing to understand: **a partial reload still hits your route closure.**

When polling fires with `only: ['usersOnline']`, Laravel re-executes the entire route handler — your controller method, all your queries, everything. Inertia only **filters** the response down to the requested keys before sending.

That means if you write:

```
// ❌ Bad – totalRevenue is computed on every poll, even though we never send it.
return Inertia::render('Dashboard', [
  'usersOnline' => User::onlineCount(),
  'totalRevenue' => Order::sum('total'), // runs on every 2s poll
]);
```

...then every poll runs both queries. The `Order::sum('total')` is wasted work. Wrapping in a closure defers the computation until Inertia decides whether the prop is being sent:

```
// ✅ Good – totalRevenue closure is never invoked during the partial reload.
return Inertia::render('Dashboard', [
  'usersOnline' => fn () => User::onlineCount(),
  'totalRevenue' => fn () => Order::sum('total'),
]);
```

This closure-wrap pattern shows up in *every* Inertia 2 feature, not just polling. Treat it as the default style for any prop that involves a database query.

GOTCHA

Inertia stops polling automatically when the browser tab is hidden, and resumes when you tab back. This is usually what you want. Override with `'keepAlive: true'` only when polling is critical (e.g., a chat-style UI).

Trade-offs & when NOT to use it

- Don't poll faster than your users can perceive change. 2–5s is plenty for most "live" UIs.
- Don't poll endpoints that hit external APIs or generate load proportional to user count.
- If the data is genuinely real-time (chat, collaborative editing), use WebSockets/Echo, not polling.

Cheatsheet

```
// Minimal idiomatic usage
usePoll(2000, { only: ['notifications', 'unreadCount'] })

// Manual control
const { start, stop } = usePoll(2000, { only: ['data'] }, { autoStart: false })
// later: start() / stop()
```

Episode 2 — Prefetching

What problem it solves

Most navigation in a SPA is predictable. A user hovering over "Settings" is overwhelmingly likely to click it. Why wait until the click to fetch the page? Inertia 2 lets you warm a client-side cache on hover (or mount), so the actual click renders instantly.

Core API surface

```
<Link
  href="/settings"
  prefetch           <!-- shorthand for prefetch="hover" -->
  :cache-for="['30s', '1m']" <!-- [fresh, stale] tuple -->
>
  Settings
</Link>
```

Three trigger modes for the `prefetch` attribute:

Value	When the fetch fires
<code>"hover"</code> (default)	On <code>mouseenter</code> , after a small delay (~75–100ms) to avoid wasted fetches from cursor sweeps
<code>"mount"</code>	When the <code><Link></code> is rendered. Use for high-priority "next step" links above the fold
<code>"click"</code>	Effectively disables prefetch; same as default Inertia behavior

`cacheFor` accepts a single duration string (e.g. `'30s'`) or a **tuple** `[fresh, stale]`:

- During the **fresh** window, a click reuses the cached response with no network.
- During the **stale** window, the cached response is rendered immediately *and* a background fetch refreshes it — the classic stale-while-revalidate pattern.
- After the stale window, the cache entry is discarded.

Annotated code walkthrough

```
<script setup>
import { Link } from '@inertiajs/vue3'
</script>

<template>
  <nav>
    <!-- Default: hover prefetch, ~30s cache -->
    <Link href="/dashboard" prefetch>Dashboard</Link>

    <!-- Mount prefetch: warm cache immediately, useful for top nav -->
    <Link href="/profile" prefetch="mount">Profile</Link>

    <!-- Fresh for 30s, then stale-while-revalidate for 1 minute -->
    <Link
      href="/reports"
      prefetch
      :cache-for="['30s', '1m']"
    >
      Reports
    </Link>
  </nav>
</template>
```

You can also prefetch imperatively:

```
import { router } from '@inertiajs/vue3'

router.prefetch('/reports', {}, { cacheFor: '1m' })
```

Conceptual model

A prefetch entry has a small state machine:

```
(no entry) —hover→ [fetching] —response→ [fresh] —fresh expires→
[stale] —click→ (serve cached + refetch in background) → [fresh]
[stale expires] → (entry deleted)
```

The ~75–100ms hover delay exists because the alternative — firing on the first mouseenter pixel — would prefetch every link the cursor brushes past while moving across the screen. The delay lets Inertia distinguish "moving toward the link" from "passing over the link."

The fresh/stale tuple matters because cache invalidation is a real problem. A pure 30s cache means an edit you just made might not show up if you navigate away and back within the window. The stale tier gives you the best of both: instant render, then a silent refresh that fixes any drift.

NOTE

The exact hover threshold and default cache duration are not stable contract — Jeffrey ballparks them at 75–100ms and 30s respectively. Treat both as implementation details that may change between releases.

Trade-offs & when NOT to use it

- Don't `prefetch` every link in a nav. You'll DDoS your own backend with cursor-sweep noise.
- Don't `prefetch="mount"` on dozens of links per page — it negates the lazy nature of Inertia visits.
- Avoid prefetching pages with **mutating side effects** in the controller (analytics writes, view counters). Prefetching counts as a visit.
- Authentication-gated pages: prefetched responses still respect auth, but a redirect-to-login flash will be cached if the user isn't signed in.

Cheatsheet

```
<!-- 90% of the time, this is all you need -->
<Link href="/users" prefetch>Users</Link>

<!-- High-traffic destination: warm cache aggressively -->
<Link href="/dashboard" prefetch="mount" :cache-for="['1m', '5m']">Dashboard</Link>
```

Episode 3 — Deferred Props

What problem it solves

You have a page where most props resolve in <50ms but one prop — say a "recent activity feed" that joins five tables — takes 800ms. With a classic Inertia visit, the user stares at a blank page for 800ms because the response is pending. With deferred props, Inertia sends the fast props immediately and resolves the slow ones in a second HTTP request, letting you render a skeleton in the gap.

Core API surface

Server side:

```

use Inertia\Inertia;

return Inertia::render('Dashboard', [
    'fastData' => fn () => $this->fastQuery(),

    // Deferred: returned in a follow-up request after the page renders.
    'activity' => Inertia::defer(fn () => $this->slowActivityQuery()),

    // Optionally group by tag. Same tag = same follow-up request.
    'reports' => Inertia::defer(fn () => $this->reportsQuery(), 'high'),
    'analytics'=> Inertia::defer(fn () => $this->analyticsQuery(), 'low'),
]);

```

Client side:

```

<script setup>
import { Deferred } from '@inertiajs/vue3'
defineProps({ activity: Array })
</script>

<template>
  <Deferred data="activity">
    <!-- Required fallback slot rendered while the deferred request is in flight -->
    <template #fallback>
      <ActivitySkeleton />
    </template>

    <!-- Default slot rendered when the prop arrives -->
    <ActivityList :items="activity" />
  </Deferred>
</template>

```

Key arguments:

- `Inertia::defer(callable, ?string $group = null)` — second argument is an optional tag. Props sharing a tag are resolved in the same follow-up request; props with different tags are resolved in parallel requests.
- `<Deferred data="...">` — the `data` prop accepts a string (single prop name) or an array (multiple prop names; the fallback shows until *all* of them resolve).
- `#fallback` slot is required. Omitting it logs a console warning and renders nothing — a surprising failure mode worth memorizing.

Annotated code walkthrough

```
// app/Http/Controllers/DashboardController.php
public function __invoke()
{
    return Inertia::render('Dashboard', [
        'user' => fn () => auth()->user(),

        // Group "high priority" data – sent in one follow-up request together.
        'stats' => Inertia::defer(fn () => $this->stats(), 'priority'),
        'recent' => Inertia::defer(fn () => $this->recent(), 'priority'),

        // Separate group – resolved in a parallel follow-up request.
        'analytics' => Inertia::defer(fn () => $this->analytics(), 'analytics'),
    ]);
}
```

```
<!-- resources/js/Pages/Dashboard.vue -->
<script setup>
import {Deferred} from '@inertiajs/vue3'
defineProps({
  user: Object,
  stats: Object,
  recent: Array,
  analytics: Object,
})
</script>

<template>
  <h1>Welcome, {{ user.name }}</h1>

  <!-- Two props, one fallback. Default slot waits for BOTH to arrive. -->
  <Deferred :data="['stats', 'recent']">
    <template #fallback>
      <div class="grid grid-cols-2 gap-4">
        <StatsSkeleton />
        <RecentSkeleton />
      </div>
    </template>
    <div class="grid grid-cols-2 gap-4">
      <StatsPanel :data="stats" />
      <RecentList :items="recent" />
    </div>
  </Deferred>

  <Deferred data="analytics">
    <template #fallback><AnalyticsSkeleton /></template>
    <AnalyticsChart :data="analytics" />
  </Deferred>
</template>
```

Conceptual model

When a route returns deferred props, Inertia does this:

1. The first response includes the non-deferred props and a manifest listing which prop names are deferred and which group they belong to.
2. The page renders. `<Deferred>` components see their target prop is missing and show the fallback.
3. Inertia automatically fires **one follow-up partial-reload request per group** (parallel, not sequential).
4. Each follow-up response arrives with its props. `<Deferred>` swaps fallback → default slot.

Why grouping matters: if you defer five props and group them all into one tag, you get one follow-up request with the longest of the five resolution times. If you split them into five tags, you get five parallel requests — better wall-clock time if your server can handle the concurrency, worse if those requests contend on the same resources (DB connection pool, etc.). Default is one group per `defer()` call without a tag, which means one parallel request per prop.

GOTCHA

Don't defer trivially fast props. Each defer group is an extra HTTP round trip — overhead alone is often 50–200ms. If your query runs in 30ms, deferring makes the page *slower*, not faster.

Trade-offs & when NOT to use it

- Use defer when a prop genuinely takes >200ms and the rest of the page is useful without it.
- Don't use defer for above-the-fold critical content. Users see the fallback first.
- Watch your group count. Too many groups → too many parallel DB queries → connection pool exhaustion.

Cheatsheet

```
// Server
'slow' => Inertia::defer(fn () => $expensiveQuery()),
'a' => Inertia::defer(fn () => $a(), 'group1'),
'b' => Inertia::defer(fn () => $b(), 'group1'), // same request as 'a'
```

```
<!-- Client -->
<Deferred data="slow">
  <template #fallback><skeleton /></template>
  <Content :data="slow" />
</Deferred>
```

Episode 4 — Lazy Loading with `optional` and `<WhenVisible>`

What problem it solves

Deferred props ship in a *follow-up* request automatically. Sometimes you don't want that — you want a prop that is **never** sent until the client explicitly asks. Examples:

- A "recommended products" carousel that lives below the fold. Why fetch it if the user never scrolls?
- A "load more comments" button that only triggers on click.
- A modal whose data isn't needed until the user opens it.

This is **client-driven** laziness, where the server says "I have this prop available, but I won't compute it unless you ask," and the client decides when to ask.

Core API surface

Server side — `Inertia::optional()`:

```
return Inertia::render('Product', [  
  'product' => $product,  
  'related' => Inertia::optional(fn () => Product::related($product)->get()),  
]);
```

`Inertia::optional()` is the modern replacement for Inertia 1's `Inertia::lazy()` (the name changed; behavior is the same). The closure is **only invoked if the prop is named in the visit's `only` array**.

Client side — `<WhenVisible>`:

```
<WhenVisible  
  data="related"                <!-- prop(s) to request -->  
  :buffer="200"                 <!-- pixels before viewport entry (rootMargin)  
-->  
  :always="false"               <!-- fire once, or every time it enters -->  
  :params="{ /* extra query data */ }"  
>  
  <template #fallback>  
    <RelatedSkeleton />  
  </template>  
  
  <RelatedProducts :items="related" />  
</WhenVisible>
```

Important options:

- `data` — prop name (string) or names (array) to request via partial reload.

- **buffer** — distance in pixels before the element enters the viewport. Effectively a **rootMargin** on the underlying IntersectionObserver. Lets you start loading before the user technically sees the element.
- **always** — if **false** (default), the request fires once and the component never observes again. If **true**, it fires every time the element re-enters the viewport (essential for infinite scroll, see Episode 5).
- **params** — extra data merged into the query string of the partial reload. Useful for "load page 2" patterns.
- **#fallback slot** — rendered until the data arrives.

Annotated code walkthrough

```
// routes/web.php
Route::get('/products/{product}', function (Product $product) {
    return Inertia::render('Products/Show', [
        'product' => $product,

        // Server prepares this lazily – only runs if client asks for it.
        'related' => Inertia::optional(
            fn () => Product::query()
                ->where('category_id', $product->category_id)
                ->where('id', '!=', $product->id)
                ->limit(8)
                ->get()
        ),
    ]);
});
```

```

<!-- resources/js/Pages/Products/Show.vue -->
<script setup>
import { WhenVisible } from '@inertiajs/vue3'
defineProps({ product: Object, related: Array })
</script>

<template>
  <article>
    <h1>{{ product.name }}</h1>
    <p>{{ product.description }}</p>
  </article>

  <!-- The browser fires a partial reload for `related` 200px before this
       section enters the viewport. If the user never scrolls down, the
       server never runs the related-products query. -->
  <WhenVisible data="related" :buffer="200">
    <template #fallback>
      <p>Loading related products...</p>
    </template>

    <RelatedGrid :items="related" />
  </WhenVisible>
</template>

```

Conceptual model

`<WhenVisible>` is a thin wrapper around the browser's [IntersectionObserver](#) API. Under the hood it roughly does:

```

const observer = new IntersectionObserver(([entry]) => {
  if (entry.isIntersecting) {
    router.reload({ only: [props.data], data: props.params })
    if (!props.always) observer.disconnect()
  }
}, { rootMargin: `${props.buffer}px` })

observer.observe(elementRef.value)

```

Two important distinctions:

- `Inertia::optional()` vs `Inertia::defer()`: Both delay computing a prop. `defer` triggers a follow-up request **automatically**. `optional` does **nothing on its own** — it requires a client-side trigger (`<WhenVisible>`, a manual `router.reload({ only: ['x'] })`, polling with `only`, etc.).
- `<WhenVisible>` vs `<Deferred>`: `<Deferred>` waits for an automatic follow-up request that Inertia fires. `<WhenVisible>` waits for the element to enter the viewport, then fires the request itself.

A useful mental shortcut:

	Server-led (auto)	Client-led (on demand)
Wrapper component	<code><Deferred></code>	<code><WhenVisible></code>
Server helper	<code>Inertia::defer()</code>	<code>Inertia::optional()</code>

GOTCHA

Without ``always: true``, `` fires exactly once and stops observing. If you want it to refire on every viewport entry — e.g., for infinite scroll where the trigger sentinel is reused — you must set ``:always="true"``. We'll see why this matters next.

Trade-offs & when NOT to use it

- Don't lazy-load above-the-fold data. The flash of fallback is worse than a slightly longer initial response.
- Don't lazy-load tiny props. The round-trip overhead exceeds the saved query time.
- Watch SEO. Crawlers may not trigger viewport-based loading. Lazy-loaded content is essentially invisible to most bots.

Cheatsheet

```
// Server: prepare but don't execute
'related' => Inertia::optional(fn () => $expensiveQuery()),
```

```
<!-- Client: fire once when visible -->
<WhenVisible data="related" :buffer="200">
  <template #fallback><Skeleton /></template>
  <Content :items="related" />
</WhenVisible>
```

Episode 5 — Infinite Scrolling

What problem it solves

You want a list that grows as the user scrolls. The classic React-flavored solution: hand-roll a scroll listener, track page numbers in state, debounce, append. With Inertia 2 you can express the same thing declaratively by composing three primitives you already know: Laravel pagination,

`<WhenVisible>`, and `Inertia::merge()`.

Core API surface

Server side — pagination + merge:


```
return Inertia::render('Posts/Index', [
  'posts' => Inertia::merge(
    fn () => Post::latest()->paginate(10)
  ),
]);
```

`Inertia::merge()` tells the client: "When a partial reload returns this prop, **merge** the new value into the existing one rather than replacing it." For arrays, merge means concat (append). For objects, it's a deep merge.

Client side:

```
<WhenVisible
  :always="!reachedEnd"
  :params="{ page: nextPage }"
  data="posts"
>
  <template #fallback>
    <p>Loading more...</p>
  </template>
</WhenVisible>
```

There is no default slot here — when the data arrives, it gets merged into the page's existing `posts` prop, so the parent template re-renders naturally.

Annotated code walkthrough (current state: the workaround)

There's a real-world wrinkle. As of Inertia 2.0.3, the client-side deep-merge implementation **does not correctly merge Laravel's paginator object** (which is a nested structure: `{ data: [...], current_page, last_page, links: [...], meta: {...}, next_page_url, ... }`). Each merge would replace the inner arrays cleanly but the surrounding paginator metadata would get clobbered or duplicated.

The pragmatic workaround Jeffrey demonstrates: **flatten the paginator to a plain array of items, and pull the pagination metadata out into sibling props that don't need merging.**

Server:

```
// routes/web.php
Route::get('/posts', function () {
    $page = request()->integer('page', 1);
    $paginator = Post::latest()->paginate(10, ['*'], 'page', $page);

    return Inertia::render('Posts/Index', [
        // The actual records – merged on each follow-up request.
        'posts' => Inertia::merge(fn () => $paginator->items()),

        // Pagination state – replaced (not merged) on each request, which is
correct.
        'page' => $paginator->currentPage(),
        'lastPage' => $paginator->lastPage(),
    ]);
});
```

Client:

```

<script setup>
import { computed } from 'vue'
import { WhenVisible } from '@inertiajs/vue3'

const props = defineProps({
  posts: { type: Array, default: () => [] },
  page: Number,
  lastPage: Number,
})

// Stop firing once we've fetched the final page.
const reachedEnd = computed(() => props.page >= props.lastPage)

// The page number to request next.
const nextPage = computed(() => props.page + 1)
</script>

<template>
  <ul>
    <li v-for="post in posts" :key="post.id">
      {{ post.title }}
    </li>
  </ul>

  <WhenVisible
    :always="!reachedEnd"
    :params="{ page: nextPage }"
    data="posts"
    :buffer="400"
  >
    <template #fallback>
      <p v-if="!reachedEnd">Loading more posts...</p>
      <p v-else>You've reached the end.</p>
    </template>
  </WhenVisible>
</template>

```

Annotated code walkthrough (ideal future state)

If/when the deep-merge bug is fixed, the cleaner form would let you keep the paginator intact:

```

return Inertia::render('Posts/Index', [
  'posts' => Inertia::merge(fn () => Post::latest()->paginate(10)),
]);

```

```

<script setup>
import { computed } from 'vue'
defineProps({ posts: Object })

const reachedEnd = computed(() =>
  props.posts.current_page >= props.posts.last_page
)
const nextPage = computed(() => props.posts.current_page + 1)
</script>

<template>
  <ul>
    <li v-for="post in posts.data" :key="post.id">{{ post.title }}</li>
  </ul>
  <WhenVisible :always="!reachedEnd" :params="{ page: nextPage }" data="posts">
    <template #fallback><Spinner /></template>
  </WhenVisible>
</template>

```

WARNING

Verify the merge behavior on your installed Inertia version before relying on the ideal form. The flatten-the-paginator workaround is safe and predictable; the ideal form depends on a specific bug fix.

Conceptual model

Infinite scroll falls out naturally once you understand the three primitives:

1. `Inertia::merge()` flips the client semantics from "replace this prop" to "merge into this prop." For an array, merge = concat. The new page's items get appended to the existing items.
2. `:params` on `<WhenVisible>` lets you inject query data into the partial reload. By computing `nextPage`, the next visit hits `/posts?page=3`, Laravel's paginator picks it up, and the merge concatenates the new records.
3. `:always="!reachedEnd"` is the brake. `<WhenVisible>` normally fires once and disconnects its observer. With `always`, it re-fires on every viewport entry. By computing a boolean derived from pagination state, you toggle the observer off precisely when there's no more data — no manual disconnect, no race conditions, no "are we loading?" flags.

The interaction between these three is what makes the pattern feel declarative. State is server-owned (the paginator), the trigger is a viewport intersection, and the termination is a derived value. No imperative scroll listeners, no manual fetch dispatch.

GOTCHA

If you forget `:always="true"` (or a truthy expression), `` fires exactly once. You'll get page 2 and then nothing else. The component appears broken; it's actually behaving as documented.

Trade-offs & when NOT to use it

- Infinite scroll harms keyboard users and screen readers if there's no "load more" alternative. Consider rendering a button fallback.
- Search engines: same caveat as lazy loading — content past the first page is essentially invisible to crawlers.
- For long lists, virtualize the rendered DOM (vue-virtual-scroller) or you'll degrade the page as items accumulate.

Cheatsheet

```
// Server
'posts' => Inertia::merge(fn () => Post::latest()->paginate(10)->items()),
'page' => request()->integer('page', 1),
'lastPage' => Post::count() / 10, // or paginator->lastPage()
```

```
<!-- Client -->
<WhenVisible
  :always="!reachedEnd"
  :params="{ page: page + 1 }"
  data="posts"
/>
```

4. Cross-Cutting Concepts

Partial reloads (`only`)

Every Inertia 2 feature ultimately funnels through a partial reload. The mechanism is unchanged from Inertia 1.x: pass `only: ['propA', 'propB']` to a visit and the response payload is filtered to those keys. The server still re-runs the full controller — the `only` filter happens at the response-building stage, *not* at the route-handler stage.

This is why **closure-wrapping props is so important across all features**. Without closures, the values are computed eagerly and discarded — work done for nothing.

The shared router options object

`usePoll`, `router.reload`, `router.visit`, `<WhenVisible>` reload behavior, and prefetch all accept the same option shape:

```
{
  only: ['prop1', 'prop2'],
  except: ['prop3'],
  preserveScroll: true,
  preserveState: true,
  onStart: () => {},
  onSuccess: (page) => {},
  onError: (errors) => {},
  onFinish: () => {},
}
```

Once you know one, you know them all. The lifecycle callbacks in particular (`onStart` / `onFinish`) are the right hook for showing custom loading indicators.

Server-driven vs client-driven laziness

A common confusion is when to use `defer()` vs `optional()`. Use this rule:

- `defer()` when you *always* want the prop eventually, but not in the first response. You're saying "send the page now, send this prop ASAP after."
- `optional()` when you *might never* want the prop. You're saying "have this ready if the client asks, otherwise don't bother."

Polling and infinite scroll naturally use `optional` patterns (the client decides when to fetch).

Skeleton-driven dashboards naturally use `defer` (the server decides everything will eventually arrive).

Caching vs merging

These are orthogonal:

- **Prefetch caching** stores a *complete page response* keyed by URL. On a real visit, the cached payload is reused instead of fetching.
- `merge()` changes how a *single prop* is combined with the existing prop value when an Inertia visit lands.

You can combine them — a prefetched response that includes a `merge()` prop will be cached, and the cached version will be merged on render — but the two features don't compete.

5. Build-Along Capstone — Course Dashboard

Build a Laravel + Inertia + Vue 3 dashboard for an online learning platform that exercises every feature in this guide.

Required surfaces

- **GET /dashboard** — Top of page: user info, today's "currently enrolled" count (polls every 5s). Middle: a "recent activity" feed (deferred). Below the fold: "recommended courses" (lazy, only when scrolled into view). Bottom: paginated "all courses" list (infinite scroll).
- **GET /courses/{course}** — Course detail page, prefetched on hover from the dashboard.

Suggested order of implementation

1. **Skeleton page.** Render `/dashboard` with hardcoded props. No Inertia features yet.
2. **Polling.** Wrap the live "enrolled count" prop in `usePoll(5000, { only: ['enrolledCount'] })` and a closure on the server.
3. **Deferred.** Move "recent activity" to `Inertia::defer()`. Add a `<Deferred>` wrapper with a skeleton fallback. Group it under the tag `'feed'`.
4. **Lazy.** Wrap "recommended courses" server-side in `Inertia::optional()`. Client side, render inside `<WhenVisible data="recommended" :buffer="300">`.
5. **Infinite scroll.** Build the paginated course list with `Inertia::merge()` + flattened paginator + `<WhenVisible :always="!reachedEnd" :params="{ page: nextPage }">`.
6. **Prefetch.** On the dashboard's course list, add `prefetch` to each `<Link>` to `/courses/{id}`. Tune `cacheFor` to `['1m', '5m']`.

By the end, a cold load of the dashboard should:

- Render `user`, `enrolledCount`, and pagination chrome immediately.
- Show skeletons for `activity` (deferred) and `recommended` (lazy-on-scroll).
- Refresh `enrolledCount` every 5 seconds.
- Append courses as the user scrolls.
- Make subsequent navigation feel instant via prefetch.

6. Glossary

- **Inertia visit** — A request/response cycle initiated by clicking a `<Link>` or calling `router.visit/reload/...`. Returns a JSON payload identifying the component, props, URL, and asset version.
- **Prop** — A key/value pair returned from the server, mapped 1:1 onto Vue component props.
- **Partial reload** — An Inertia visit that requests a subset of props via the `only` array; server runs the full handler, response is filtered.

- **Deferred prop** — A prop wrapped in `Inertia::defer()`. Skipped in the initial response, resolved in an automatic follow-up request.
- **Optional prop** — A prop wrapped in `Inertia::optional()`. Never sent unless the client explicitly names it in an `only` array. (Replaces Inertia 1's `Inertia::lazy()`.)
- **Prop merging** — Behavior enabled by `Inertia::merge()`: incoming values are combined with existing prop values rather than replacing them. Arrays concat; objects deep-merge.
- **Stale-while-revalidate** — Cache strategy where a stale entry is served immediately while a background fetch refreshes it. Implemented via the `[fresh, stale]` tuple in `cacheFor`.
- **IntersectionObserver** — Native browser API for observing when an element enters or exits the viewport (or a specified root). `<WhenVisible>` wraps it.
- **Tuple** — A fixed-length, positional array. Inertia uses a 2-tuple `[fresh, stale]` for `cacheFor`, where position determines meaning.
- **Laravel paginator object** — The serialized shape returned by `->paginate()`: `{ data, current_page, last_page, per_page, total, links, meta, next_page_url, prev_page_url, path, from, to }`. `data` is the array of records for the current page; everything else is pagination metadata.

7. Open Questions / Things to Verify

These items are stated in the source material but are version-dependent or imprecisely documented. Confirm against the current Inertia release notes before relying on them in production.

1. **Hover threshold for `prefetch`**. Stated as roughly 75–100ms. Not part of the public API; subject to change.
2. **Default `cacheFor` duration**. Stated as ~30 seconds. The exact default and whether it differs between `prefetch` modes (`hover` vs `mount`) should be checked.
3. **Deep-merge bug for paginator objects (Inertia 2.0.3)**. Workaround in this guide flattens the paginator. Confirm whether a later release (2.0.4+) fixes the nested merge so the cleaner shape is viable.
4. **Component import paths and casing**. The components are referenced as `<Deferred>` and `<WhenVisible>` (PascalCase) from `@inertiajs/vue3`. Verify the canonical capitalization in your installed version's exports.
5. **`<Deferred>` missing-fallback behavior**. Stated as "logs a console warning, renders nothing." Confirm whether this throws in newer versions or remains a soft warning.
6. **`keepAlive` polling behavior**. Default is to pause when the tab is hidden. Verify the exact event hook (Page Visibility API vs blur/focus) if you have edge cases like PiP video or background tab notifications.

7. `<WhenVisible>` **re-observation after** `always` **re-fires**. Whether each re-trigger uses a fresh observer or the same one matters if you're holding references for cleanup.
-

End of notes.